

SOLID 원칙은 객체 지향 프로그래밍(OOP)에서 유지보수하기 쉽고, 확장이 용이하며, 읽기 좋은 코드를 작성하기 위한 5가지 설계 원칙이다.

1. 단일체계원칙(single responsibility principle)

-각 class는 하나의 임무와 목적을 가진다

하나의 class가 여러 임무를 맡으면 test를 하여 어디가 문제인지 알아내기 어렵고, class 속 내용 중에서 필요한 부분만 빼서 쓰기 까다로워진다.

```
class ReportGenerator {  
    public String generateReport() {  
        // 보고서 생성 로직  
    }  
}  
  
class ReportSaver {  
    public void saveReport(String report) {  
        // 보고서 저장 로직  
    }  
}  
  
class ReportPrinter {  
    public void printReport(String report) {  
        // 보고서 출력 로직  
    }  
}
```

이처럼 class를 역할마다 나누어두면 보기도 편하고 나중에 부분부분 수정하기에 좋다.

2. 개방/폐쇄원칙(open closed principle)

-각 class는 확장에는 열려있고, 변경에는 닫혀 있어야한다.

소프트웨어 개발 작업에 이용된 많은 모듈 중에서 하나를 수정할 때, 다른 모듈을 수정해야한다면 번거롭다. 개방폐쇄 원칙은 시스템의 구조를 올바르게 조직하여 나중에 수정할 때 기존의 코드를 변경하지 않아도, 코드를 추가함으로써 기능의 추가나 변경이 가능하다.

```

abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

class Triangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Triangle");
    }
}

```

기능을 확장하면서 기존 코드를 변경하지 않으려면 추상화를 이용한다. 예를 들면, 여러 종류의 도형을 그리는 코드를 설계할 때, ocp원칙에 따라 shape라는 class를 정의하고, 각 도형은 shape class를 상속받아 자신만의 draw메서드를 구현한다.

추상화: 복잡한 내부 원리나 구체적인 구현 내용은 숨기고, 사용자에게 필요한 핵심 기능(공통된 틀)만 겉으로 드러내는 것

3. 리스코프 치환 원칙(liskov substitution principle)

-자식 class는 언제나 부모 클래스를 대체할 수 있어야한다

만약 부모 class에서 잘 수행하는 내용이 자식 class에서 수행되지 않으면 해당 내용을 독립된 인터페이스에 두고, 수행할 수 있는 자식 class에 인터페이스를 적용하는 방식으로 해야한다.

```

class Bird {
    public void fly() {
        System.out.println("새가 날아갑니다.");
    }
}

class Penguin extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("펭귄은 날 수 없습니다.");
    }
}

```

여기서 부모클래스 bird는 날 수 있고, 펭귄 class가 새 class를 상속하였다. 하지만 펭귄은 부모 class에 있는 fly가 불가능하므로 이는 잘못된 설계이다.

4. 인터페이스분리원칙(interface segregation principle)

-class는 자신이 사용하지 않을 메소드를 구현하도록 강요받지 않는다

하나의 큰 인터페이스보다는 여러 개의 작은 인터페이스로 분리하는 것이 좋다.

```

interface Worker {
    void work();
}
interface Eater {
    void eat();
}
class Human implements Worker, Eater {
    public void work() { System.out.println("사람이 일합니다."); }
    public void eat() { System.out.println("사람이 밥을 먹습니다."); }
}
class Robot implements Worker {
    public void work() { System.out.println("로봇이 작업합니다."); }
}

```

worker와 eater 인터페이스를 분리하여 robot 클래스가 eat를 하지 않아도 된다.

-isp가 잘 적용된 예

5. 의존성 역전 원칙(dependency inversion principle)

-고수준 모듈이 저수준 모듈에 의존해서는 안된다. + 둘 다 추상화에 의존해야 한다.

```

class Lamp {
    public void turnOn() {
        System.out.println("Lamp is on");
    }

    public void turnOff() {
        System.out.println("Lamp is off");
    }
}

class Button {
    private Lamp lamp;

    public Button(Lamp lamp) {
        this.lamp = lamp;
    }

    public void press() {
        if (/* some condition */) {
            lamp.turnOn();
        } else {
            lamp.turnOff();
        }
    }
}

```

버튼 class가 램프 class를 제어할 때, 이 경우는 버튼 class가 램프 class에 강하게 결합되어 있으므로 dip원칙을 위반하고 있다.

```

interface Switchable {
    void turnOn();
    void turnOff();
}

class Lamp implements Switchable {
    @Override
    public void turnOn() {
        System.out.println("Lamp is on");
    }

    @Override
    public void turnOff() {
        System.out.println("Lamp is off");
    }
}

class Button {
    private Switchable device;

    public Button(Switchable device) {
        this.device = device;
    }

    public void press() {
        if (/* some condition */) {
            device.turnOn();
        } else {
            device.turnOff();
        }
    }
}

```

dip원칙을 적용하면 버튼 class는 램프class가 아니라 switchable이라는 추상화된 인터페이스에 의존해야한다.

이러면 버튼 class는 램프class 뿐 아니라 fan, tv와 같은 다른 장치들도 제어할 수 있게 된다.